

修了試験A-4

合計点 38/63 点 ?

合格点 85%以上 (54以上/63)

0 / 0 ポイント

受験者情報の登録

登録メールアドレス *

htanaka@777.so-net.jp

御名前 *

田中 秀伯

修了試験問題

38 / 63 ポイント

✓ 再帰型ニューラルネットワークに関する文章のうち、明らかに誤っている文章を以下のうちから1つ選べ。 *1/1

- ☐ 勾配爆発を防止する観点から考えた場合、再帰型ニューラルネットワークに用いる活性化関数は、ReLUよりもシグモイド関数やTanh関数が適している。
- ☒ 再帰型ニューラルネットワークでは、入力長が長くなると勾配爆発や勾配消失が起きやすくなるが、入力長を途中で区切ることは精度を落としてしまうため、行われることはない。 ✓
- ☐ 再帰型ニューラルネットワークを時間方向に展開し、一般的な順伝播型のニューラルネットワークと同じように誤差逆伝播計算を行うことを、BPTTとよぶ。
- ☐ 再帰重み行列の最大特異値が1より小さい状態で時間方向の計算を繰り返すと、長い系列では勾配が減衰しやすくなる。



RNNは時系列データを処理するモデルとして知られているが、構造が再帰的なため、指数関数的に勾配が蓄積してしまい、誤差逆伝播時に勾配が爆発してしまう可能性が比較的高いという問題があげられる。そのため、爆発を起こしにくい（ あ ）が活性化関数として選ばれやすい。また、（ い ）が閾値を超えた際に制限を加える（ う ）が有効な手段として用いられることが多い。

✓ （ あ ） に当てはまる選択肢について、以下から1つ選べ。 * 1/1

- ☐ Tanh関数 や ReLU関数
- ☒ Tanh関数 や Sigmoid関数
- ☐ ReLU関数 や ELU関数
- ☐ ReLU関数 や Sigmoid関数



✗ （ い ） に当てはまる選択肢について、以下から1つ選べ。 * 0/1

- ☒ L1ノルム
- ☐ スパース
- ☐ エラスティックネット
- ☐ L2ノルム



✓ （ う ） に当てはまる選択肢について、以下から1つ選べ。 * 1/1

- ☐ 勾配ブースティング
- ☐ 重み最大マッチング
- ☒ 勾配クリッピング
- ☐ 重み共有



開発初期には、シンプルな再帰型ニューラルネットワークを利用していくことで、時系列データが処理できることが明らかになったが、同時に様々な課題が浮かび上がった。これらの課題を解決するための方法として、LSTMやGRUのようなゲート付き再帰型ニューラルネットワークが提案された。ゲート付き再帰型ニューラルネットワークの提案によって、課題が克服され、長い間時系列データを扱う上では、デファクトスタンダードとして扱われてきた。

また、その他にも双方向RNNや有向グラフィカルモデルとしての回帰結合型ネットワークなども開発された。

✓ 双方向RNNの記述として誤っているものを選び。 *

1/1

- ☐ 双方向RNNのもとになるRNNには、LSTMやGRUなどのゲート付きRNNも利用される。
- ☐ 必ずしも双方向RNNの方が、RNNよりもよいとはいえない。
- ☐ 双方向RNNの開発によって、自然言語処理の分野は改良が進んだ。
- ☒ 株価のような時系列データに対して特に双方向RNNは使われる。 ✓





*

1/1

入力 x 、1つ前の隠れ層の出力 h^{t-1} 、バイアス B として時刻 t における隠れ層での線型結合を $A_2 = h^{t-1}W_h + xW_x + B$ とする。このとき、リカレントニューラルネットワークの逆伝播 BPTT における勾配式を考える。入力と隠れ層における重みパラメータをそれぞれ とすると

$$dW_h = (\text{あ}), \quad dW_x = (\text{い})$$

が成り立つ。(あ)(い)に当てはまる組み合わせについて最も適切なものを1つ選べ。

$$\begin{aligned} (\text{あ}) &: (h^{t-1})^T \cdot dA_2 \\ (\text{い}) &: (x)^T \cdot dA_2 \end{aligned}$$

☒ A


$$\begin{aligned} (\text{あ}) &: (h^{t-1})^T \cdot dA_2 \\ (\text{い}) &: dA_2 \cdot (x)^T \end{aligned}$$

☐ B

$$\begin{aligned} (\text{あ}) &: dA_2 \cdot (h^{t-1})^T \\ (\text{い}) &: (x)^T \cdot dA_2 \end{aligned}$$

☐ C

$$\begin{aligned} (\text{あ}) &: dA_2 \cdot (h^{t-1})^T \\ (\text{い}) &: dA_2 \cdot (x)^T \end{aligned}$$

☐ D


- ✓ リカレントニューラルネットワークの拡張版として双方向リカレントニューラルネットワークがある。リカレントニューラルネットワークと比較した、双方向リカレントニューラルネットワークの特徴について、適切なものを1つ選べ。 *1/1
- ☐ 順方向の時間ステップで系列データの入力を行うことに加えて、逆方向の系列データを加算してモデルへの入力とすることで、未来の情報も踏まえた予測を可能にしている。
- ☐ 順方向と逆方向でそれぞれ入力を行い、並行して処理することによって計算効率性を高めたモデルであるため、通常のリカレントニューラルネットワークよりも計算速度が速い。
- ☐ 順方向で処理した隠れ層の状態と逆方向で処理した隠れ層の状態をそれぞれ共有することで、精度向上を目指したモデルだが、モデルの入力方法や出力形式を調整する必要がある。
- ☒ 未来の情報も踏まえる予測を可能とするため、精度の向上が期待されるが、モデルの構造上適用できるタスクが制限されるデメリットがある。 ✓

- ✓ LSTM (Long Short-Term Memory) が持つゲート機構の1つとして、忘却ゲートがある。時間ステップを t 、メモリセル c 、入力データ x 、隠れ層の状態 h として、忘却ゲートを通した長期記憶であるメモリセル $\tilde{c}$ を示した式について、適切な選択肢を1つ選べ。また、活性化関数として、 σ はシグモイド関数、 $\tanh$ はTanh関数を表している。 *1/1

$$\tilde{c} = c_{t-1} \circ \sigma \left(W^F \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \right)$$

☒ A


$$\tilde{c} = c_{t-1} \circ W^F \circ \tanh \left(\begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \right)$$

☐ B

$$\tilde{c} = c_{t-1} \circ \sigma \left(W^F \begin{bmatrix} h_t \\ x_{t-1} \end{bmatrix} \right)$$

☐ C

$$\tilde{c} = c_{t-1} \circ \sigma \left(W^F \circ \tanh \left(\begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \right) \right)$$

☐ D


✕ RNN において 長期依存性を解決したモデルとして、LSTM (Long Short-Term Memory) と GRU (Gated Recurrent Unit) がある。それぞれのモデルにはゲート機構と呼ばれる構造が隠れ層に配置されている。それぞれのモデルが持つゲート機構について、適切な選択肢を 1 つ選べ。

- ☒ LSTM が持つゲート機構の 1 つとして、入力ゲートがある。入力ゲートは 1 つ前の隠れ状態における出力のみを元に、どれだけ長期記憶に情報を加えるかの割合を算出する役割を持つ。 ✕
- ☐ LSTM が持つゲート機構の 1 つとして、リセットゲートがある。リセットゲートは 1 つ前の隠れ状態における出力とその時点の入力を元に、過去の状態をどれだけ引き継ぐかの割合を考える役割を持つ。
- ☐ GRU が持つゲート機構の 1 つとして、更新ゲートがある。更新ゲートは過去の情報のうち、どれだけ将来に渡す必要があるかを判断する役割を持つ。
- ☐ GRU が持つゲート機構の 1 つとして、出力ゲートがある。出力ゲートは他のゲートで求めた割合を用いてどの程度を出力とし、その残りを次の隠れ層に引き渡す役割を持つ。

開発初期には、シンプルな再帰型ニューラルネットワークを利用していくことで、時系列データが処理できることが明らかになったが、同時に様々な課題が浮かび上がった。これらの課題を解決するための方法として、LSTMやGRUのようなゲート付き再帰型ニューラルネットワークが提案された。ゲート付き再帰型ニューラルネットワークの提案によって、課題が克服され、長い間時系列データを扱う上では、デファクトスタンダードとして扱われてきた。

また、その他にも双方向RNNや有向グラフィカルモデルとしての回帰結合型ネットワークなども開発された。

✓ 再帰型ニューラルネットワークに関する記述として最も適切なものを1つ選べ。 *1/1

- ☐ 再帰型ニューラルネットワークは、全結合型ニューラルネットワークに比べ勾配消失や勾配爆発の問題が起きにくい。
- ☐ 再帰型ニューラルネットワークを時間方向に展開し、順伝播ネットワークに書き換えて誤差逆伝播計算を行うことをバックプロパゲーション・スルー・タイムといい、BPTTでは通常の誤差逆伝播計算とは異なり二回微分を用いる。
- ☐ 自然言語のような入力長が毎回異なる可能性があるデータを扱う場合は、再帰型ニューラルネットワークよりも、全結合型ニューラルネットワークのほうが向いている。
- ☒ 勾配爆発を防止するという視点で考えると、再帰型ニューラルネットワークに用いる活性化関数はReLUよりもシグモイド関数やtanh関数が適している。 ✓

✓ ゲート付き再帰型ニューラルネットワークに関する記述として最も適切なものを1つ選べ。 *1/1

- ☒ LSTMおよびGRUにおけるゲートの活性化関数には、通常、シグモイド関数が用いられる。 ✓
- ☐ GRUは、LSTMよりもゲート数が多いという特徴がある。
- ☐ ゲート付き再帰型ニューラルネットワークは、通常のシンプルな再帰型ニューラルネットワークに比べ長期依存性を学習しにくい。
- ☐ GRUは、3つのゲート（入力ゲート、リセットゲート、更新ゲート）を持つ。

ゲート付きRNNの代表的なモデルとして、LSTMがある。LSTMは、役割の異なる3つのゲートと、長期にわたる情報を保持するための記憶セルを備えている。3つのゲートは、入力ゲート、出力ゲート、忘却ゲートである。これらのゲートの値を求めるために用いられる活性化関数は（ あ ）である。

入力ゲート、出力ゲート、忘却ゲートの出力を、それぞれI、O、Fとする。また、本問では、時刻tにおける入力 x_t と、前時刻の隠れ状態 h_{t-1} から生成される候補セル状態を、

$$x_t W_x^{(g)} + h_{t-1} W_h^{(g)}$$

と表す。なお、簡略化のため、候補セル状態の生成に用いる活性化関数とバイアス項は省略する。このとき、時刻tにおける記憶セルの状態 C_t を表す式は（ い ）となる。

また、 $\odot$ は、2つの行列またはベクトルの対応する要素同士の積であるアダマール積を表す。各ゲートの計算に記憶セルの値を利用して、ゲートの制御を行う結合をピープホール結合という。前時刻の記憶セルの状態を C_{t-1} 、更新後の記憶セルの状態を C_t とすると、忘却ゲートと入力ゲートの計算には C_{t-1} を、出力ゲートの計算には C_t を用いる。

✓ （ あ ） に当てはまる選択肢について、以下から1つ選べ。 *

1/1

- ☐ Tanh関数
- ☐ 恒等写像関数
- ☐ ReLU関数
- ☒ Sigmoid関数



× (い) に当てはまる選択肢について、以下から1つ選べ。 *

0/1

$$I \odot (x_t W_x^{(g)} + h_{t-1} W_h^{(g)}) + O \odot F \odot C_{t-1}$$

☒ A

×

$$I \odot (x_t W_x^{(g)} + h_{t-1} W_h^{(g)}) + F \odot C_{t-1}$$

☐ B

$$O \odot (x_t W_x^{(g)} + h_{t-1} W_h^{(g)}) I \odot + F \odot C_{t-1}$$

☐ C

$$I \odot x_t W_x^{(g)} + O \odot h_{t-1} W_h^{(g)} + F \odot C_{t-1}$$

☐ D

× LSTMに関する文章のうち、明らかに誤っている文章を以下のうちから1つ選べ。 *0/1

- ☐ LSTM の特定時刻における出力は、記憶セルの値を活性化関数Tanhに通し、出力ゲートとのアダマール積をとったものとなる。
- ☐ LSTM において、前時刻の記憶セルの情報を弱める際には、記憶セルと忘却ゲートの行列積を取る。
- ☒ LSTM の歴史は古く、1990年代に提唱された。 ×
- ☐ LSTMにおいて、CEC (Constant Error Carousel) は記憶セルの自己ループ構造を指し、忘却ゲートはその情報保持の度合いを制御する。



2014年にKyunghyun Cho（조 경현）らによって発表されたゲート付きRNNがGRUである。LSTMよりも少ないゲート数にすることで、パラメータを減少させることに成功した。

GRUにおいて、（ あ ）ゲートは過去の隠れ状態とのアダマール積を取った後、活性化関数（ い ）を通すことで新たな隠れ状態を算出する。（ う ）ゲートは過去の隠れ状態と新たな隠れ状態の混合割合を決定する。

× （ あ ）に当てはまる選択肢について、以下から1つ選べ。 * 0/1

- ☐ 出力
- ☒ 更新
- ☐ リセット
- ☐ 忘却

×

× （ い ）に当てはまる選択肢について、以下から1つ選べ。 * 0/1

- ☐ ReLU
- ☐ Softmax
- ☒ Sigmoid
- ☐ Tanh

×



× (う) に当てはまる選択肢について、以下から1つ選べ。 * 0/1

☒ リセット

×

☐ 忘却

☐ 更新

☐ 出力

× 従来型のSeq2Seqでは、Encoderが入力系列の情報を単一の固定長コンテキストベクトルへ圧縮するため、入力系列が長くなると情報を十分に保持できない場合がある。その課題を克服するために開発されたアテンション機構における説明文として適切なものを以下のうちから1つ選べ。 *0/1

☒ アテンション機構は、全ての入力データに等しい重みを割り当て、復号化器がすべてのデータを均等に扱うようにする。 ×

☐ アテンション機構は、復号化器が固定長の文脈ベクトルを使用して出力を生成するために使われる。

☐ アテンション機構は、seq2seqモデルで入力系列の重要な部分に焦点を当てることで、復号化器のパフォーマンスを向上させる。

☐ アテンション機構は、復号化器が入力系列の最初と最後の要素に特に注目するように設計されている。

✓ Seq2Seqにおけるエンコーダとデコーダ間の情報伝達を強化する技術として、アテンション機構がある。ここでは、デコーダの隠れ状態とエンコーダの各隠れ状態の内積に基づいてスコアを算出する、ドット積型アテンションを考える。ドット積型アテンションに関する記述として、最も適切なものを1つ選べ。 *1/1

- ☐ コンテキストベクトルは、ソフトマックス関数によって正規化されたスコアを用いて、エンコーダの隠れ状態の単純な和として計算される。
- ☒ アテンションスコアは、デコーダの隠れ状態とエンコーダの隠れ状態のドット積に基づいて計算される。 ✓
- ☐ ソフトマックス関数による正規化は、各スコアに対して個別に適用される。
- ☐ コンテキストベクトルは、アテンションスコアに基づいて計算され、デコーダの隠れ状態に依存しない。

✓ Seq2Seqでは、Decoderへの入力として本来は直前に予測した値を使用するが、学習初期には誤った予測が多く、学習が不安定になる場合がある。そのため、学習時に正解データをDecoderの次の入力として与える（ あ ）を用いることで、学習を安定させる。 *1/1

また、従来型のSeq2Seqでは、Encoderが入力系列の情報を単一の固定長コンテキストベクトルへ圧縮してDecoderへ渡すため、入力系列が長くなるほど情報を十分に保持できない場合がある。この情報ボトルネックを緩和するため、Decoderが出力時刻ごとにEncoderの各隠れ状態を重み付けして参照するAttention機構が提案された。

（ あ ）に当てはまる語句として、最も適切なものを1つ選べ。

- ☐ カリキュラム学習
- ☒ 教師強制 ✓
- ☐ 構造出力
- ☐ 協調フィルタリング

✓ Attentionに関する記述として適切でないものを1つ選べ。 *

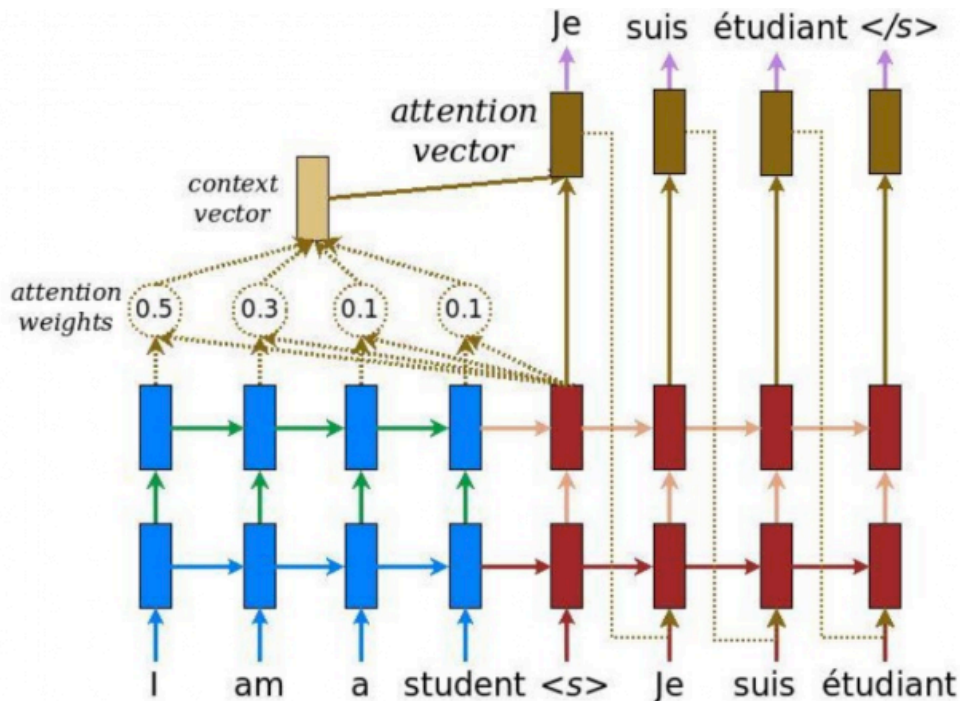
1/1

- ☐ Attentionでは、Encoderの最後の隠れ層だけでなく、すべての隠れ層の状態ベクトルをDecoderに伝える。
- ☐ 入力と出力の対応関係を学習させているとみなせる。
- ☒ Attentionは、時系列データで用いられる機構であり、画像データでは用いられない。 ✓
- ☐ Encoderから受け取った情報に対して、Decoderで重みづけすることで重要度を付ける。



✕ 以下の図は、RNN の学習において良く用いられる Teacher Forcing と呼ばれる手法である。この手法では、学習時に正解系列をデコーダーの入力および出力に用いる手法である。 *0/1

この手法の問題点として、適切な選択肢を 1 つ選べ。



https://www.tensorflow.org/tutorials/text/nmt_with_attention

引用：

- ☐ 学習および収束が遅くなる。
- ☒ 評価時の分布と学習時の分布が同じとなり、精度が過大に評価されてしまう。 ✕
- ☐ 双方向RNN には適用できない。
- ☐ 評価時はデコーダーが出力した系列がデコーダーの入力となるため、評価時の分布と学習時の分布が異なる。

✕ 2017年、Googleが発表した論文「Attention is all you need」で *0/1
Transformer と呼ばれるモデルが提案された、このモデルは、RNN
(Recurrent Neural Network) のように系列データに対して処理を行うモ
デルであるが、既存のモデルよりも飛び抜けた精度を達成したため、一
躍有名となった。Transformer に対する説明文として、最も適切なものを
1つ選べ。

- ☐ 自然言語処理におけるベクトルの作成として文字埋め込みがあるが、Transformer
では代わりに位置埋め込みを採用している。
- ☒ アテンションと呼ばれる機構を中心に組み立てられた深層学習モデルであり、 ✕
RNN と同様に逐次的な学習を必要とする。
- ☐ アテンション機構での処理は、すべて単一の情報を入力としており、異なる系列デ
ータ間や系列データ内の単語間の関係性も学習できる。
- ☐ TransformerはSelf-Attentionを中心に構成されたモデルであり、自然言語処理だけ
でなく、画像認識などのタスクにも応用されている。

- ✕ Transformerは、系列を逐次的に処理する再帰層や畳み込み層を用いず、Self-Attentionと位置ごとのフィードフォワードネットワークを中心に構成されている。この構造により、系列内の各位置を並列に処理しやすい。 *0/1

Scaled Dot-Product Attentionでは、Query、Key、ValueをそれぞれQ、K、Vとし、QueryとKeyの内積に基づいて、Valueをどの程度参照するかを表す重みを求める。

本問では、Q、K、Vはそれぞれ線形変換を適用した後の値であるものとする。また、選択肢中の「 $Q \cdot K$ 」はQとKの転置との積を簡略的に表したものであり、 d_q はQueryとKeyの共通の次元数を表すものとする。

Attention(Q, K, V)を表す式として、最も適切なものを1つ選べ。

$$\text{softmax} \left(\frac{Q \cdot K}{\sqrt{d_q}} \right) V$$

☐ A

$$\text{sigmoid} \left(\frac{Q \cdot K}{d_q} \right) V$$

☐ B

$$\text{softmax} \left(\frac{Q \cdot V}{d_q} \right) K$$

☒ C

✕

$$\text{sigmoid} \left(\sqrt{\frac{Q}{d_K}} K \right) V$$

☐ D

NLPの世界を大きく変えたモデルに、論文「Attention is all you need」で発表されたTransformerがある。Transformerは、それまでNLPの世界で主流であったRNN（及びCNN）を利用するモデル構造をやめ、Attentionのみを用いてEncoder-Decoder型のモデルを設計した。Transformerは当時の最高精度を更新し、NLPのデファクトスタンダードになった。Transformerの成功により、Transformerを双方向RNNのように利用したBERTがGoogleから発表され、大きな話題となった。BERTの事前学習として二値分類と多クラス分類が教師なし学習によって行われる。

✓ 以下の文章のうち、誤った内容の文章をひとつ選べ。 *

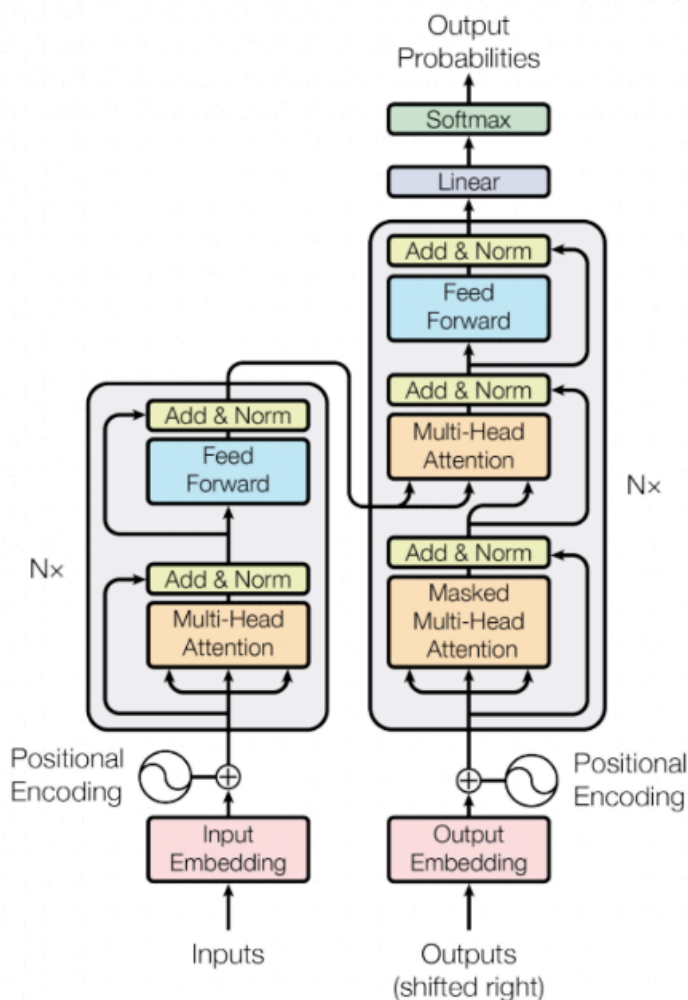
1/1

- ☐ Transformerには3つのAttention構造が含まれる。エンコーダにあるのはSelf-Attention、デコーダにあるAttentionのうち入力側にあるのがSelf-Attention、出力側にあるのがSource-Target-Attentionである。
- ☐ Attentionは、RNNやCNNと組み合わせて利用することができ、自然言語処理から画像認識まで、様々なタスクに適応可能な汎用性の高い構造である。
- ☒ Transformerで使われているAttention構造は、Scaled Dot-Product Attentionとも呼ばれている。これはAttentionの計算をする際、softmax関数を通す前の値に $1/\sqrt{d_k}$ (d_k はkeyの次元)をかけることでスケールする処理であり、この処理は勾配が爆発することを防ぐ効果がある。 ✓
- ☐ Transformerで利用されているSource-Target-Attention内で行われている計算について、QueryはDecoder、KeyはEncoder、ValueはEncoderから伝達された値を使う。

時系列データを扱うニューラルネットワークモデルとして、畳み込みニューラルネットワーク(CNN)やリカレントニューラルネットワーク(RNN)が主に用いられてきた。しかし、2017年に“Attention is All You Need”(A. Vaswani, et al.)と題した論文が発表されて以降、時系列データを扱う際に、畳み込み層や再帰結合層を廃してAttentionのみを用いる手法が注目されている。“Attention is All You Need”内で提案されたAttentionのみを用いたモデルはTransformerと呼ばれる。

✓ Transformer のアーキテクチャの説明として、不適切な選択肢を 1 つ選べ。 *1/1

以下の図は Transformer のアーキテクチャを表している。Transformer は Encoder と Decoder で構成されており、それぞれ N 個の同一の層から成る。



- ☐ Encoder ・ Decoder は共に順伝播ネットワークを含んでいる。
- ☒ Decoder を構成する N 個の層は全て Positional Encoding の出力を入力として受け取る。 ✓
- ☐ Encoder の最終出力は Decoder の Multi-Head Attention に入力される。
- ☐ Decoder では Masked Multi-Head Attention が使用されている。

- ✓ Transformerは、これまで自然言語モデルで使用されていた畳み込み層やリカレント層を Self-attention層に置き換えることで、計算速度を向上させた。

Self-attention層が畳み込み層やリカレント層と比較し優れている点として、不適切な選択肢を 1 つ選べ。

- ☐ Self-Attentionでは、系列内の離れた2つの位置も1層内で直接情報をやり取りできるため、長距離の依存関係を捉えやすい。
- ☐ 各単語に対応する隠れ層の直前の隠れ層全てが Self-attention によって参照されるため、離れた単語同士の依存関係を学習しやすい。
- ☐ Self-Attentionでは、系列内の各位置を同時に処理できるため、RNNと比較して並列計算を行いやすい。
- ☒ 畳み込み層やリカレント層は固定長のシーケンスしか扱えなかったのに対し、 Self-attention 層は可変長のシーケンスを扱える。 ✓

- ✓ 以下の表は、英独翻訳タスクEN-DEと英仏翻訳タスクEN-FRにおける翻訳の質を測る BLEU スコアと各モデルの学習コストを Transformer と他の自然言語モデルで比較したものである。 *1/1

表から読み取れることとして、適切な選択肢を 1 つ選べ。

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

- ☐ Transformer (big) は、EN-DEにおいてGNMT + RL EnsembleよりもBLEUスコアが低く、学習コストも高い。
- ☐ Transformer (base model) は、EN-FRにおいてConvS2SよりもBLEUスコアが高い。
- ☒ Transformer (big) は EN-FR において最も BLEU スコアが高い。 ✓
- ☐ Transformer (base model) の EN-FR におけるトレーニングコストは、Deep-Att + PosUnk モデルよりも高い。

- ✓ 複数のScaled Dot-Product Attentionを並列に実行し、それぞれの出力を結合する仕組みをMulti-Head Attentionという。Multi-Head Attentionは、TransformerのEncoderとDecoderの両方で使用される。EncoderのSelf-Attentionでは、Query、Key、Valueはいずれも（ あ ）から、それぞれ異なる線形変換によって生成される。 *1/1

（ あ ）に当てはまる文章として最も適切な選択肢を以下から1つ選べ。

- ☐ Encoder全体の最終出力
- ☒ そのEncoder層への入力
- ☐ Decoder全体の最終出力
- ☐ 直前のDecoder層の出力



- ✗ デコーダにおけるマルチヘッド・アテンションは、2種類ある。デコーダの入力から見て1つ目のマルチヘッド・アテンションはセルフ・アテンションであるが、ここでは、予測すべき単語の情報が予測時に利用されないようにするためにマスクがかけられる。 *0/1

マスクがかけられた場所の値は、ソフトマックス関数に入力される前に、（ い ）に置き換えられる。

（ い ）に当てはまる数として最も適切な選択肢を以下から1つ選べ。

- ☐ $-\infty$
- ☐ -1
- ☐ 1
- ☒ 0



✕ デコーダの入力から見て2つ目のマルチヘッド・アテンションでは、(*0/1
う) から伝わってくる。

(う) に当てはまる語句として最も適切な選択肢を以下から1つ選
べ。

- ☐ Key と value はエンコーダ、query はデコーダ
- ☐ Value はエンコーダ、query と key はデコーダ
- ☒ queryはエンコーダ、keyとvalue はデコーダ
- ☐ query と keyはエンコーダ、valueはデコーダ

✕

以下のコードを踏まえて、設問に答えよ。

```
class ScaledDotProductAttention(nn.Module):
    def __init__(self, scale_factor):
        super().__init__()
        self.scale_factor = scale_factor

    def forward(self, q, k, v):
        dk = k.size(-1)
        # スケールされた内積を計算します
        scores = torch.matmul(q, k.transpose(-2, -1)) / torch.sqrt(dk
* self.scale_factor)
        attention = (あ)(scores, dim=-1)
        output = (い)(attention, v)
        return output
```

✓ プログラム内（ あ ）（ い ）について、以下のうち正しい選択肢 *1/1 を1つ選べ。

- ☒ （あ） torch.softmax （い） torch.matmul ✓
- ☐ （あ） torch.add （い） torch.matmul
- ☐ （あ） torch.matmul （い） torch.softmax
- ☐ （あ） torch.softmax （い） torch.add



以下のコードを踏まえて、設問に答えよ。

```
class MultiheadAttention(nn.Module):
    def __init__(self, num_heads, input_size, head_size):
        super().__init__()

        self.num_heads = num_heads
        self.head_size = head_size
        # 各Query、Key、Valueに対する線形変換を定義します
        self.q_linear = nn.Linear(input_size, num_heads * head_size)
        self.k_linear = nn.Linear(input_size, num_heads * head_size)
        self.v_linear = nn.Linear(input_size, num_heads * head_size)

        self.attention = ScaledDotProductAttention(scale_factor=1.0 /
head_size)
        self.fc = nn.Linear(num_heads * head_size, input_size)

    def forward(self, q, k, v):
        bs = q.size(0)

        # 線形変換
        q = self.q_linear(q).view(bs, -1, self.num_heads,
self.head_size).transpose(1,2) # (bs, num_heads, q_len, head_size)
        k = self.k_linear(k).view(bs, -1, self.num_heads,
self.head_size).transpose(1,2) # (bs, num_heads, k_len, head_size)
        v = self.v_linear(v).view(bs, -1, self.num_heads,
self.head_size).transpose(1,2) # (bs, num_heads, v_len, head_size)

        # Scaled Dot-Product Attention
        attention_scores = (う) # (bs, num_heads, q_len, head_size)

        attention =
attention_scores.transpose(1,2).contiguous().view(bs, -1,
self.num_heads * self.head_size)
        output = self.fc(attention)
        return output
```



✓ プログラム内（ う ）について、以下のうち正しい選択肢を1つ選べ。 *1/1

- ☐ self.attention(q, v, k)
- ☒ self.attention(q, k, v)
- ☐ self.attention(k, q, v)
- ☐ self.attention(v, k, q)



✗ 画像認識におけるデータ拡張は、設定された問題に対して有効な拡張手法を割り当てることで汎化性能をさらに向上することができる。例えば、オクルージョンや遮蔽が頻繁に起こるような撮影状況に対して特に有効なデータ拡張について、以下のうちから1つ選べ。 *0/1

- ☐ Random Erase
- ☐ Random Rotate
- ☒ Random Brightness
- ☐ MixUp



✓ ドローンなどから撮影された画像では、機体の姿勢や撮影方向の変化によって、対象物がさまざまな角度で写る場合がある。このような回転方向の変化に対する頑健性を高めるために、最も直接的なデータ拡張手法を1つ選べ。 *1/1

- ☐ Random Erase
- ☐ Random Brightness
- ☐ MixUp
- ☒ Random Rotate



学習データに変化を加えて、データ、量・種類を拡張させることで汎化性能を向上しようとする手法をデータ拡張という。画像データに対するデータ拡張では、画像を回転させたり、画像の明るさなどを変えるなどのデータ拡張が行われる。

データ拡張の手法の一つに、2つの画像を合成して新しいサンプルを作成する（あ）という手法がある。モデルの学習時にはラベル側も混ぜ合わせる必要があり、この画像の合成の結果により、正則化の効果が生まれ、かつ、画像の中間も識別できるようになるため、精度が向上するといわれている。

✕ データ拡張に関する説明文として適切なものを選び。 *

0/1

- ☐ データ拡張にはデメリットがないため、積極的に行うことが望ましい。
- ☒ データ量が十分にある場合には、データ拡張を行う必要はない。 ✕
- ☐ データ拡張は音声データに対しても行われる。
- ☐ 学習コストをさげるためにも、データ拡張は行われる。

✓ （あ）に当てはまる選択肢について、以下から1つ選び。 *

1/1

- ☒ Mixup ✓
- ☐ Contrast
- ☐ RandomCrop
- ☐ Random Flip

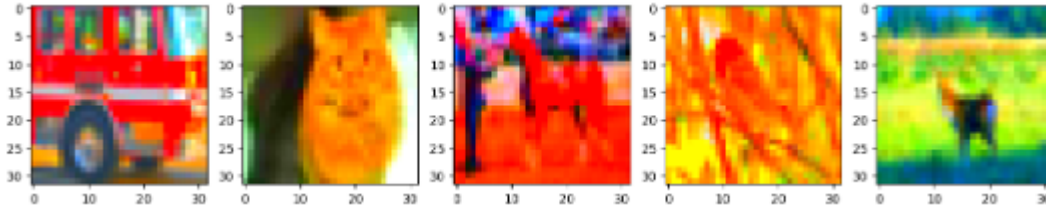
以下のコードを踏まえて、設問に答えよ。

```
from torchvision.transforms import v2

transform = v2.Compose([
    v2.ToTensor(), # 画像をテンソルに変換
    v2.ColorJitter(contrast=0.3, brightness=0.2, saturation=10, hue=0.001) # 色の変更
])
```



- ✓ 画像は、プログラムを実行しデータ拡張された cifar-10 画像を出力したものである。引数の値が不適切なため、画像の彩度が不自然に高いか低いものが含まれている。どのように訂正すればよいか、以下のうち正しい選択肢を以下から 1 つ選べ。 *1/1



- ☐ 'v2.ColorJitter' の引数 'contrast' の値を (0.3, 0.9) に変更する
- ☒ 'v2.ColorJitter' の引数 'saturation' の値を 0.5 に変更する ✓
- ☐ 'v2.ColorJitter' の引数 'hue' の値を 0 に変更する
- ☐ 'v2.ColorJitter' の引数 'brightness' の値を 0.3 に変更する

以下のコードを踏まえて、設問に答えよ。

```
def mixup(X, y, alpha):
    batch_size = X.size()[0]
    indices = torch.randperm(batch_size)
    # ラムダをサンプリング
    lam = torch.tensor([np.random.beta(alpha, alpha)]).to(X.device)
    # 入力Xのmixup処理
    X_mix = lam * X + (1 - lam) * X[(あ), :]
    # ラベルyのmixup処理
    y_mix = lam * y + (1 - lam) * y[(い)]
    return X_mix, y_mix
```

alpha = 0.9

- ✓ 画像は、プログラムを実行しデータ拡張された cifar-10 画像を出力したものである。プログラム内の（ あ ）、（ い ）に入る正しい組み合わせを以下から選べ。 *1/1

ただし、yはone-hot形式に変換されたクラスラベルであるものとする。



- ☐ （あ）X （い）y
- ☒ （あ）indices （い）indices
- ☐ （あ）indices （い）y
- ☐ （あ）X （い）indices



- ✓ 深層学習における過学習を防ぐなどの対策手法として正規化が挙げられる。正規化は（ あ ）を用いてデータを加工する手法であり、バッチごとに（ あ ）を算出してデータを加工する手法をバッチ正規化という。 *1/1

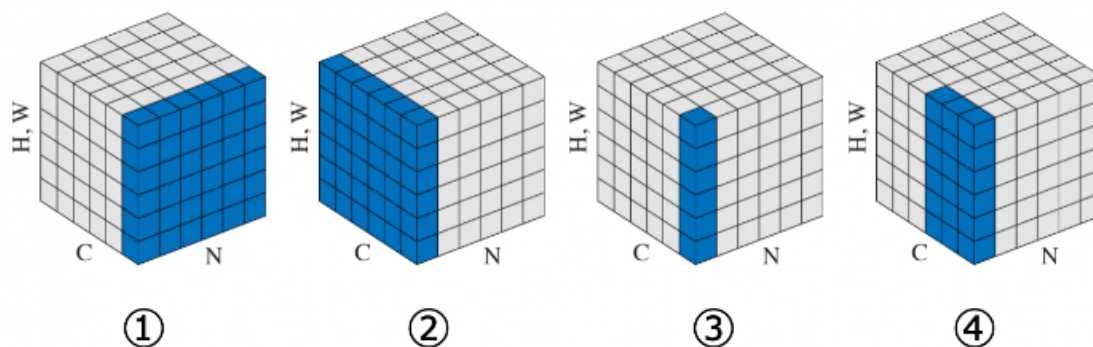
（ あ ）に当てはまる用語について、以下のうちから最適なものを1つ選べ。

- ☐ 平均と四分位数
- ☐ 合計と分散
- ☐ 最大値と最小値
- ☒ 平均と標準偏差



✕ Batch Normalizationでは、バッチサイズが小さい場合にバッチ統計量の推定が不安定になり、性能が低下することがある。そこで、バッチサイズに依存しない正規化手法としてGroup Normalizationが用いられることがある。 *0/1

グループ正規化を行う際に参照するデータを示す図として、以下のうちから適切なものを1つ選べ。



- ☒ ①
- ☐ ②
- ☐ ③
- ☐ ④

✕

汎化性能を向上させるための方法の一つとして、正規化が行われる。ニューラルネットワークに対して行われる代表的な正規化手法としては、Instance NormalizationやBatch Normalization などがある。

✓ Instance Normalizationに関する記述として、誤っているものを1つ選べ。 *1/1

- ☒ 各サンプル・各チャンネルについて、画像の高さ方向だけを用いて平均と分散を算出する。 ✓
- ☐ スタイル変換やテクスチャ生成などのタスクで用いられることがある。
- ☐ 各サンプル・各チャンネルについて、高さ方向と幅方向の要素から平均と分散を算出する。
- ☐ 学習時だけでなく、推論時にも各入力から統計量を算出して正規化を行う。

✓ Batch Normalizationは、ミニバッチ内のデータ数が（ あ ）場合に、 *1/1
安定した統計量を算出しやすい。また、Batch Normalizationは（ い ）
する効果があることや、初期値への依存を低減し、比較的大きな学習率
を利用しやすくする。

（ あ ）、（ い ）に当てはまる用語の組み合わせとして、以下のうちから最適なものを1つ選べ。

- ☐ （あ）少ない （い）過剰適合を抑制
- ☐ （あ）少ない （い）ロバスト性を向上
- ☐ （あ）多い （い）ロバスト性を向上
- ☒ （あ）多い （い）過剰適合を抑制 ✓

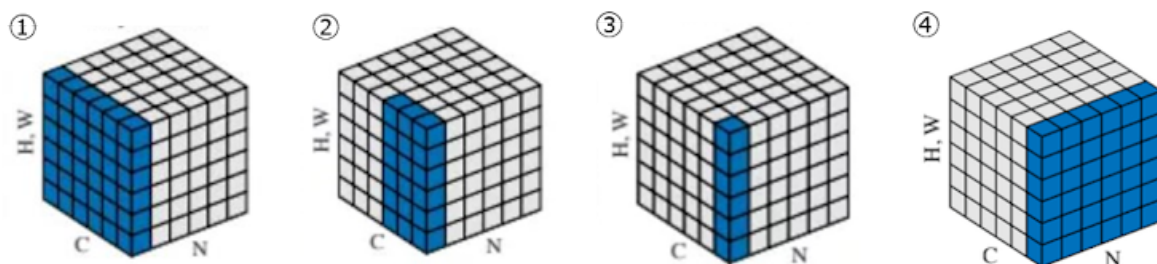
✓ Batch Normalizationに関する説明文として適切なものを選び。 *

1/1

- ☐ Batch Normalizationは過学習を抑制する効果があるが、学習率は低く抑える必要がある。
- ☒ Batch Normalizationは、重みの初期値への依存を低減し、学習を安定させる効果が期待できる。 ✓
- ☐ Batch Normalizationはミニバッチごとのデータ数が少ない場合にも有効な正規化方法である。
- ☐ Batch Normalizationは、テスト時にも訓練時と同様に入力データの平均と標準偏差を求めて正規化を行う。

汎化性能を向上させるための方法の一つとして、正規化が行われる。ニューラルネットワークに対して行われる代表的な正規化手法としては、Instance NormalizationやBatch Normalization などがある。

以下の図は各Normalizationの該当箇所を示したものである。バッチノーマライゼーションは（ あ ）、レイヤーノーマライゼーションは（ い ）、インスタンスノーマライゼーションは（ う ）、グループノーマライゼーションは（ え ）で表される。



✓ (あ) ~ (え) に当てはまる選択肢について、適切な選択肢を 1 つ選
べ。 *1/1

- ☐ (あ) ① (い) ④ (う) ③ (え) ②
- ☐ (あ) ④ (い) ① (う) ② (え) ③
- ☐ (あ) ① (い) ④ (う) ② (え) ③
- ☒ (あ) ④ (い) ① (う) ③ (え) ②



✓ Batch Normalizationはニューラルネットワークにおいて、学習時間の短
縮や初期値への依存低減、過学習の抑制など効果がある。しかし (あ) にはバッチ統計量の推定が不安定になり、性能が低下することがある。そのため、Group NormalizationやLayer Normalizationなど、バッチサイズに依存しない正規化手法が用いられることがある。 *1/1

(あ) に当てはまる用語について、以下のうちから最も適切な選択肢を 1 つ選べ。

- ☐ weightの初期値の分散が小さい場合
- ☒ ミニバッチ当たりのデータ数が少ない場合
- ☐ 層の数が少ない場合
- ☐ 学習データの量が多い場合



✓ BatchNormalization層はその内部にバッチ単位や全体を通しての平均・分散を保持しており、これらは学習、検証、テストのそれぞれで適切に使い分ける必要がある。この使い分けについて、適切な組み合わせを選択肢から1つ選べ。 *1/1

- ☐ [学習時]バッチ統計量, [検証時]バッチ統計量, [テスト時]全体の移動統計量
- ☐ [学習時]全体の移動統計量, [検証時]全体の移動統計量, [テスト時]バッチ統計量
- ☒ [学習時]バッチ統計量, [検証時]全体の移動統計量, [テスト時]全体の移動統計量 ✓
- ☐ [学習時]全体の移動統計量, [検証時]バッチ統計量, [テスト時]バッチ統計量

✗ モデルのバイアスを小さくするために選択すべきアンサンブル手法として、以下のうち正しい選択肢を1つ選べ。 *0/1

- ☐ ブートストラップ
- ☐ ブースティング
- ☒ バギング ✗
- ☐ スタッキング

✗ モデルのバリエーションを小さくするために選択すべきアンサンブル手法として、以下のうち正しい選択肢を1つ選べ。 *0/1

- ☒ ブートストラップ ✗
- ☐ バギング
- ☐ スタッキング
- ☐ ブースティング

アンサンブル学習とは、集合知を活用した学習器のことで、1つの学習器から得られる予測よりも複数の学習器の予測を利用した方が、最終的に良い予測を得られることが多いことから活用される手法である。(あ) は、平均法の一つで、重複ありでランダムなデータの部分集合を取り出す。

✓ (あ) に当てはまる選択肢について、以下から1つ選べ。 * 1/1

- ☒ バギング法 ✓
- ☐ ランダム部分空間法
- ☐ ランダムパッチ法
- ☐ ペースティング法

✓ 以下のうちから、アンサンブル学習として誤っている説明文を1つ選べ。 *1/1

- ☐ アンサンブル学習は弱学習器を利用するが、複数の深層学習モデルを組み合わせることもある。
- ☐ ランダムフォレストは平均法の一つで、決定木を複数集め、各決定木の予測結果を多数決、または平均を取るモデルである。
- ☐ 大きく、平均法、ブースティング法、スタッキング法に分けられる。
- ☒ ハード投票分類器は個別の分類器の確率を平均し、最も確率の高いクラスを予測クラスとして返す。 ✓

モデルの汎化性能を向上させる方法の一つとして、複数のモデルの結果を組み合わせるアンサンブル学習とよばれるものがある。アンサンブル学習のなかでも代表的な手法として、平均法やブーティング法とよばれる手法が存在する。ブーティングは、学習器が（ あ ）型なので、平均法に比べて計算コストが高い。そのことで、モデルの（ い ）を下げる効果がある。（ う ）はブーティングのひとつで、イテレーション毎にインスタンスの重みを調整するのではなく、新予測器を前の予測器の残差に適合させようとする。（ え ）は、複数のモデルの予測結果をメタモデルの入力として利用し、最終的な予測を行う手法である。

✓ （ あ ）に当てはまる選択肢について、以下から1つ選べ。 * 1/1

- ☒ 直列
- ☐ 教師あり
- ☐ 並列
- ☐ 教師なし



✗ （ い ）に当てはまる選択肢について、以下から1つ選べ。 * 0/1

- ☒ バリエーション
- ☐ 汎化誤差
- ☐ ノイズ
- ☐ バイアス



× (う) に当てはまる選択肢について、以下から1つ選べ。 *

0/1

- ☐ ランダムフォレスト
- ☒ アダブースト
- ☐ XGブースト
- ☐ 勾配ブースティング

×

× (え) に当てはまる選択肢について、以下から1つ選べ。 *

0/1

- ☐ ブースティング法
- ☒ 平均法
- ☐ スタッキング法
- ☐ ブートストラップ法

×

× ハイパーパラメータと、モデルの性能や計算資源との関係として正しいものを、以下の選択肢から1つ選べ。 *0/1

- ☐ 学習率が大きいと、収束が遅くなり、計算時間が増加する。
- ☐ 隠れ層やユニットの数が多いほど、過学習のリスクが高くなる。
- ☒ バッチサイズが小さいほど、学習率の影響が大きくなる。
- ☐ バッチサイズが多いほど、計算時間が増加し、メモリの要求量が低減する。

×



✓ ハイパーパラメータの探索手法のうち、指定された範囲のハイパーパラメータの値全ての組み合わせを試す手法はどれか。 *1/1

- ☐ ビームサーチ
- ☐ ベイズ最適化
- ☐ ランダムサーチ
- ☒ グリッドサーチ



✗ プログラム内（ あ ）では、nn.Dropout の引数を p=0.3 としている。 *0/1
この引数について、以下のうち正しい選択肢を 1 つ選べ。

```
class DropoutNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        # ドロップアウトを定義します。
        self.dropout = nn.Dropout(p=0.3) ... (あ)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

- ☐ ドロップアウト率が30%でユニットが無効化される
- ☐ ネットワークの30%のユニットが訓練中に2倍の学習率で訓練される
- ☒ ドロップアウト率が30%でユニットが有効化される
- ☐ ネットワークの30%のユニットがテスト時に無効化される



機械学習モデルを制御するためにハイパーパラメータを設定する必要がある。ハイパーパラメータの一例として決定木の深さやクラス数数が挙げられる。検証用データの精度を確認しハイパーパラメータを調整することを繰り返すことで、モデルの汎化性能をより高めることができる。ハイパーパラメータの最適な値の探し方はいくつか挙げられる。



✓ 以下のうちから、それぞれの手法の説明として誤っている説明文を1つ *1/1
選べ。

- ☐ ランダムサーチはある確率分布に従って探索を行うため、良い確率分布を用いれば、グリッドサーチよりも少ない試行回数で良好なハイパーパラメータを探索できる。
- ☒ グリッドサーチとは、指定された回数だけ個々のハイパーパラメータのために ランダムな値を選び、それらを組み合わせて評価する手法である。 ✓
- ☐ あらかじめパラメータの探索候補を与え、全ての組み合わせでモデルを評価する手法をグリッドサーチという。
- ☐ ランダムサーチなどを改良することで、ハイパーパラメータを自動で調整するライブラリが定期的に開発・改良されている。
- ☐ ハイパーパラメータの最適な値の探し方として、モデルや探索条件によっては、手動での調整が適している場合もある。

ベイズ最適化は、評価に大きなコストがかかる目的関数を、代理モデルによって近似しながら効率的に最適化する手法である。代表的な代理モデルとして（あ）を用いられる。

ベイズ最適化は、1回の評価コストが（い）、探索空間の次元が（う）場合に有効であることが多い。また、次に評価する点を選ぶために獲得関数を用いる。代表的な獲得関数の一つである（え）は、現在までに得られた最良値をどの程度改善できるかの期待値に基づく。

✓ （あ）に当てはまる選択肢について、以下から1つ選べ。 * 1/1

- ☒ ガウス過程回帰 ✓
- ☐ 万能近似定理
- ☐ 条件付き確率
- ☐ モンテカルロ積分

✕ (い) に当てはまる選択肢について、以下から1つ選べ。 *

0/1

- ☒ 均一であり
- ☐ 微分可能であり
- ☐ 非常に大きく
- ☐ 非常に小さく

✕

✓ (う) に当てはまる選択肢について、以下から1つ選べ。 *

1/1

- ☒ 小さい
- ☐ 微分可能の
- ☐ 1次元の
- ☐ 大きい

✓

✕ (え) に当てはまる選択肢について、以下から1つ選べ。 *

0/1

- ☐ カーネルトリック
- ☐ EI (Expected Improvement)
- ☐ PI (Probability of Improvement)
- ☒ ベイジアンネットワーク

✕

このフォームは デロイト トーマツ ディープスクエア株式会社 内部で作成されました。
このフォームが不審だと思われる場合 [報告](#)

Google フォーム



